

Lab exercises

Part 15 —  AI Evaluation · 15-ai-evals/lab/EXERCISES.md

Lab — AI Evaluation

Work top to bottom. Each exercise has a **stated deliverable**; if you can't produce it, you haven't finished. No API keys are required — everything runs offline against the code in `../code/` or against your own reasoning on paper.

1. Feel the sample-size trap (1h)

Run `python code/statistical_rigor_for_evals.py` and read section 3's output (72% vs 74% AT `n=50` vs `n=2000`).

- **1a.** Using `wilson_ci()` directly, find the smallest `n` (to the nearest 50) at which a 71% vs. 73% accuracy gap produces **non-overlapping** Wilson 95% CIs, holding the same `agreement` correlation the script uses. Show your search.
- **1b.** Explain in two sentences why McNemar's test can reject "these are the same prompt" at a smaller `n` than the CI-overlap heuristic requires — they are not answering exactly the same question.
- **1c.** Take a real (or plausibly real) claim you've seen in a vendor deck or blog post — "our new prompt improved accuracy by 3 points" — and write the one follow-up question this section trained you to ask.

Deliverable: the `n` from 1a with your search shown, two sentences for 1b, one question for 1c.

2. p-hack yourself, on purpose (45 min)

Run `p_hacking_demo()` from the same script with `n_examples=20` instead of 50 (edit the call at the bottom of the file, or call it directly from a REPL).

- **2a.** Report the number of "significant" false wins at `n=20` vs. the script's default `n=50`. Which direction did it move, and does that match your intuition about statistical power?
- **2b.** Now set `n_variants=200` at the default `n=50`. Report how many false wins you see. What does this tell you about a team that A/B-tests one prompt against 200 LLM-generated rewordings on a fixed 50-example set and ships whichever looks best?
- **2c.** Propose a concrete fix for the team in 2b that doesn't just mean "get a bigger eval set" (that's the obvious answer note 02 §2 already gives you — propose a *process* fix instead).

Deliverable: two numbers, one paragraph of interpretation, one process fix.

3. Build and break a rubric (1.5h)

Open `code/judge_bias_and_calibration.py`. Read `make_answers()` and `run_comparison()`.

- **3a.** Add a fifth binary criterion, `cited` (does the answer reference its source), generated the same way as `grounded/complete/safe`. Wire it into both the human and judge scoring so `human_accept` and `judge_accept` require all five criteria. Re-run the headline comparison (section 2 of the script's output). Report the new kappa for the binary rubric — did adding a criterion help, hurt, or barely move it, and explain why in terms of the AND-of-criteria mechanism note 02 §1.1 describes.
- **3b.** Set `verbosity_bias=0` and `leniency_bias=0` in the call inside `main()`'s headline section (the `run_comparison(n, verbosity_bias, leniency_bias, ...)` call). Re-run. Which kappa moved more, binary or Likert, and does that match the "damage control, not immunity" claim in note 02 §1.2?
- **3c.** In your own words (3-5 sentences), explain why `position_bias_demo()` is pairwise-only while `verbosity_bias_demo()` and `leniency_bias_demo()` apply to absolute scoring. What would it take to make position bias meaningful in an absolute-scoring judge?

Deliverable: the new kappa from 3a with your explanation, the comparison from 3b, and the 3-5 sentences from 3c.

4. Calibrate a judge against real human labels (2h)

You have no LLM API access for this exercise — do the calibration workflow on paper, using 20 real examples you construct by hand.

- **4a.** Write 20 short (question, answer) pairs covering a task you know well (customer support, code review comments, whatever you have domain knowledge in). Make at least 6 of them clearly bad in a way your rubric should catch.
- **4b.** Write a 4-criterion binary rubric for this task (following note 02 §1.1's four rules). Hand-score all 20 examples yourself as the "human" label.
- **4c.** Now role-play "the judge": re-score the same 20 examples a second time, at least a day later or after doing something else for 20 minutes, without looking at your first pass. This simulates a noisy-but-not-malicious grader.
- **4d.** Compute Cohen's kappa between your two passes using `cohens_kappa()` from `code/judge_bias_and_calibration.py` (import it directly). Report the number and its Landis & Koch label.
- **4e.** If kappa is below 0.7, look at the specific examples where your two passes disagreed and diagnose *why* — an ambiguous criterion, a genuinely hard example, or you just made an error. Rewrite one rubric criterion to close the biggest source of disagreement.

Deliverable: the rubric, the kappa number, and the one rubric edit from 4e with your reasoning.

5. Design a custom eval suite (2h, written)

Pick a system from an earlier module (a RAG app from Module 08 [08-rag], an agent from Module 06 [06-ai-agents], or your own project). You have no real production traces — invent 10 plausible failure examples based on what you know about the system's weak points.

- **5a.** Sort your 10 examples into the eval pyramid (note 01 §4): which can a deterministic check catch, which need an LLM judge, which genuinely need a human. Justify each placement in one sentence.
- **5b.** For the ones needing an LLM judge, write the rubric (binary criteria, per note 02 §1.1).
- **5c.** Using the golden-dataset-sizing table in note 01 §4, state how many examples you'd want in each pyramid tier for this system to reliably catch a 3-point regression, and justify the number.
- **5d.** Sketch the CI gate: name at least one absolute floor metric and one regression-delta metric, and state the paired test you'd use for the regression delta (McNemar or paired bootstrap — which, and why).

Deliverable: the sorted 10 examples, the rubric, the sizing numbers, and the CI gate sketch.

6. Stretch: contamination detective work (1.5h, written)

Pick one benchmark from note 01 §2 (MMLU, HumanEval, SWE-bench, GPQA, or Arena).

- **6a.** Find (via a web search, if you have access, or from what you already know) one documented criticism or gaming vector specific to that benchmark, beyond what note 01 §2 already lists.
- **6b.** Propose a concrete, checkable contamination-detection test for it, following the two techniques in note 01 §2 (public-vs-fresh-set gap, verbatim-continuation probing).
- **6c.** If you were a hiring manager and a candidate's resume cited a benchmark score with no methodology details, what's the one follow-up question from this module you'd ask?

Deliverable: the criticism, the test design, and the question.